

智能纪要：AI课程项目开发框架与分工研讨

2026年7月16日

 张永威  吴紫萌 | 今天修改

会议主题：AI课程项目开发框架与分工研讨

会议时间：2026年7月16日（周四） 10:47 - 11:47（GMT+08）

参会人： 张永威 @ 吴紫萌

智能会议纪要由 AI 生成，可能不准确，请谨慎甄别后使用

总结

AI社区审核系统项目分工与技术方案讨论

- 💡 核心结论：
- 项目采用 全栈模块化分工，每人负责完整功能模块以提升效率
 - 技术栈选定为 React 前端、Python FastAPI 后端、MySQL 数据库 及 Element Plus UI
 - 产品定位为 极简 AI 审核社区，聚焦文字内容审核与申诉核心流程

项目分工与功能设计

🧑‍💻 项目分工模式

采用 全栈分工，每人负责完整功能模块

- 放弃传统 前后端分离 分工
- 每人负责完整功能模块开发
- 模块包括前端、后端、测试和文档
- 避免跨模块接口对接的复杂性

🏠 核心功能设计

构建 极简文字社区，包含审核与申诉流程

- 创建话题标题与发起人
- 用户发布文字内容至社区广场
- 采用 AI 初审与复审双 Agent 流程
- 支持用户对 AI 审核结果进行申诉

🎯 功能范围界定

聚焦核心功能，暂不扩展复杂交互

- 社区仅支持 文字发布，不处理图片
- 回复以新帖子形式呈现，不做嵌套评论
- 暂不实现点赞、关注等社交功能
- 需实现上下文压缩以处理长对话

技术选型与方案

技术栈	选型方案	最终选择与原因
前端框架	Vue（简单） vs React（流行）	React，学习新技术，增强简历竞争力
后端语言	Python（FastAPI） vs Java（Spring Boot）	Python + FastAPI，更适配 AI 原生开发
数据库	MySQL vs SQLite	MySQL，更符合生产环境实践
UI 组件	多种选择	Element Plus，基于推荐且成熟
AI 框架	LangGraph 等	采用 Python 相关 AI SDK，便于集成

内容由 AI 生成

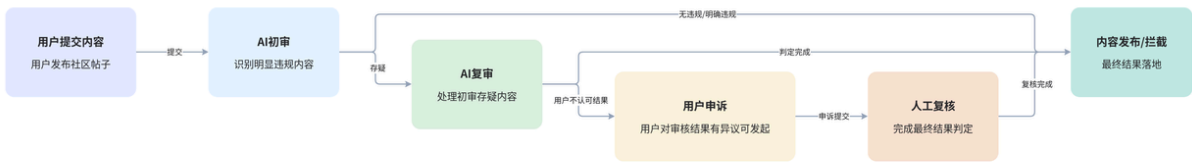
本次会议围绕课程AI审核类项目的落地需求，完成产品定位、技术选型、分工规则、功能边界的全维度确认，内容如下：

- 一、项目核心定位与产品设计方案
 - 项目核心目标
 - 高分适配要求：完整嵌套前序课程教授的全流程开发规范，严格贴合课程项目的评分标准，冲击项目最高分
 - 求职价值提升：所有技术选型均选用业界主流技术，项目落地后可直接写入求职简历，提升简历的竞争力
 - 产品落地载体：选用极简文字社区作为核心产品形态，完全匹配需求中识别违规内容的核心要求，无需求偏离
 - 产品形态设计

- 社区创建规则：每个社区由发起人创建并设置专属话题，所有发布内容按发布时间线性堆叠展示，无嵌套层级
- 交互逻辑简化：不设计嵌套评论、点赞、关注、转发等冗余交互，用户对内容的反馈直接以新帖子的形式发布
- 结构复杂度控制：线性堆叠的内容展示逻辑，不需要设计复杂的关联表结构，大幅降低数据库的设计与维护成本

核心业务流程

- 审核链路设计：用户发布内容后先经AI初审，明显违规内容直接拦截，存疑内容进入AI复审环节，最终输出审核结果
- 申诉机制设计：支持用户对审核结果发起申诉，申诉内容进入人工复核环节，完成整个审核流程的闭环
- 准确率优化设计：双阶段审核的设计，既提升审核效率，也能降低单阶段审核的误判率，提升整体审核的准确率



AI生成

AI社区审核业务全流程

功能边界确认

- 非必要功能剔除：明确不做图片上传、富文本编辑、多社区权限体系、复杂用户体系等超出开发周期的功能
- 优先级排序规则：所有功能按核心审核链路优先的原则排序，优先保障核心流程跑通，周边功能按需扩展
- 烂尾风险防控：严格控制功能范围，避免出现需求越做越复杂、最终核心功能无法交付的项目烂尾风险

二、全栈技术栈选型确认

前端技术方案

- 核心框架选型：选用 React 框架搭配 TypeScript 开发，放弃更熟悉的 Vue 框架，借项目完成新技术的学习积累
- 配套组件选型：UI 组件库采用 Element Plus，样式方案选用 Tailwind CSS，全部为当前业界主流的成熟技术栈
- 选型价值说明：放弃 Vue 框架的选择，虽然团队对 Vue 更熟悉，但 React 在大厂的应用范围更广，学习价值更高

后端技术方案

- 核心架构选型：采用 Python + FastAPI 架构，放弃 Java 技术栈，更适配 AI 能力的对接需求，开发效率更高
- 数据库选型：选用 MySQL 作为核心数据库，暂不引入微服务、Redis 等非必要组件，避免不必要的复杂度
- 非必要组件剔除：不引入微服务架构，当前项目无高并发需求，微服务只会增加不必要的复杂度，完全没有落地的必要性
- 开发流程优化：前期先用 MOCK 数据模拟 AI 审核的返回结果，核心链路跑通后再对接真实大模型接口，避免调试影响进度

◦ AI 能力设计

- workflow 架构：采用双 Agent 架构拆分初审、复审角色，对应 LangGraph 的状态流设计，可直接对接成熟开源组件
- 适配优化方案：引入轻量上下文压缩能力，适配长社区内容的审核场景，避免大模型上下文窗口溢出问题
- 扩展性设计：双 Agent 架构的设计符合当前业界 AI 工作流的主流方向，后续迭代的扩展性更强，可适配更多的业务场景

◦ 选型交付要求

- 多方案对比要求：需输出前后端、数据库、AI 框架的多方案对比文档，明确每个选型的优劣势与决策理由
- 亮点突出设计：将上下文压缩、双 Agent 工作流作为核心技术亮点，在后续文档与答辩中重点展示
- 评分点适配：多方案对比的文档是课程评分的核心考察点，必须完整覆盖所有核心组件的选型论证，不能直接拍板定案

• 三、团队分工与开发节奏安排

◦ 分工模式设计

- 分工规则确认：不按前后端拆分任务，改为按业务模块拆分，每个模块负责人独立完成对应模块的全流程工作
- 沟通成本优化：模块独立闭环的分工模式，避免跨模块对接的接口联调、规范对齐等沟通成本，适配短周期开发需求
- 传统模式弊端规避：传统前后端拆分的模式需要提前对齐所有接口规范，后续联调至少要占用1-2天的时间，完全不符合短周期的开发要求

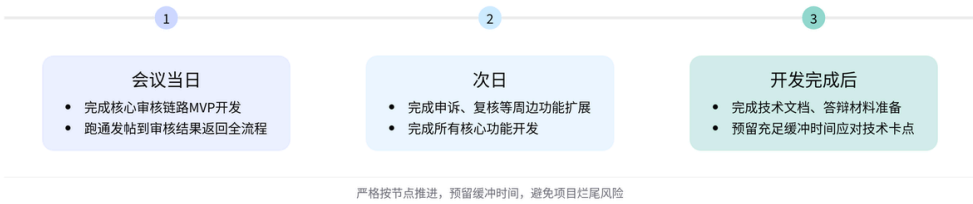
◦ 团队角色分配

- 协调角色安排：由团队内大三软件工程专业成员负责整体任务协调与进度同步，不承担过重开发任务，保障推进节奏
- 开发角色安排：其余成员根据自身技术熟悉度认领对应业务模块，所有成员均参与全栈开发，同步完成技术学习

- 协调角色合理性：大三成员的课程压力相对更小，有更充足的时间跟进整体进度，能有效避免项目推进出现无人协调的混乱情况

◦ 开发节奏规划

- MVP 开发节点：优先在会议当日完成核心审核链路的 MVP 开发，跑通从发帖到审核结果返回的全流程
- 功能扩展节点：在 MVP 跑通的基础上，于次日完成申诉、复核等周边功能的扩展，预留充足的文档与答辩准备时间
- 缓冲机制设计：预留充足的缓冲时间，即使开发过程中出现技术卡点，也不会影响最终的项目交付，保障交付的稳定性



AI 生成

项目开发节奏规划

◦ 文档管理规则

- 文档生成方式：所有需求、技术方案文档均由 AI 辅助生成初稿，再由团队成员共同确认调整，减少人工撰写成本
- 同步规则确认：所有文档、进度信息统一同步至团队群聊，所有成员可随时迭代调整，无需走复杂的审批流程
- 效率优化设计：利用 AI 工具大幅降低文档撰写的时间成本，把更多的人力投入到核心功能的开发与调试中，提升整体的开发效率

• 四、配套资源与其他共识说明

◦ 开发工具共识

- 工具选用规则：团队成员可根据个人习惯选用 VS Code 或其他 AI 辅助开发工具，无需统一强制要求，优先保障开发效率
- 代码校验规则：AI 辅助生成的代码需经人工校验正确性，避免出现逻辑漏洞，保障项目的稳定性
- 习惯适配设计：不强制统一开发工具，尊重每个成员的使用习惯，避免因为工具适配浪费不必要的时间

◦ 需求迭代规则

- 需求变更规则：后续若有新的需求点，需经团队共同确认后再纳入开发范围，严格控制需求蔓延，避免开发失控

- 优先级调整规则：所有功能的优先级可根据实际开发进度灵活调整，若进度紧张则优先保障核心功能，非核心功能直接剔除
- 节奏稳定保障：严格执行需求冻结规则，核心需求确认后不再做大的调整，保障开发节奏的稳定性

◦ 答辩准备规划

- 内容准备规则：所有功能设计、技术选型提前准备好对应的说明内容，答辩时可清晰展示设计思路与技术亮点
- 评分适配规则：所有文档与功能设计严格贴合课程的评分标准，确保所有评分点均有对应的交付内容支撑
- 遗漏风险防控：提前把所有评分点对应的交付内容整理完毕，避免答辩前临时抱佛脚，出现遗漏评分点的情况

👍 智能纪要反馈收集

待办

- ☒ 访问问题排查：排查当前服务器无法访问的原因，确认服务器运行状态及域名有效性，协助吴紫萌解决访问异常问题（来自吴紫萌）——张永威
- ☐ 规则文档整理：梳理确认社区帖子回复采用按时间顺序叠加的方案，完成对应规则文档的总结输出，需在 2 天内完成相关功能评估（来自吴紫萌）——张永威

👍 智能纪要反馈收集

相关链接

- ▶ • 文字记录